

TUTORIAL PRÁTICO

Agenda de Compromissos com Emojis de Categoria

React Native, Expo SDK 54, Expo Go e Web

Guia passo a passo

Julho de 2026

Foco: uma única tela, lista de compromissos, categorias coloridas, placar e cadastro por modal

Como utilizar este material

Este documento foi preparado como guia para conduzir uma aula prática com TypeScript, React ou React Native. As instruções indicam o que explicar, o que digitar, qual resultado esperar e como agir quando ocorrer um erro.

Objetivo do monitor

Conduzir a turma em pequenas etapas. Não peça aos alunos que copiem todo o código de uma vez. Em cada etapa, explique a ideia, digite junto com a turma, salve o arquivo e verifique o resultado.

Resultado esperado

- Aplicativo com uma lista de compromissos do dia.
- Cada compromisso mostra título, data, horário e categoria colorida.
- Placar com total de compromissos, concluídos e o próximo horário.
- Marcação de compromisso como concluído ao tocar na caixa de seleção.
- Remoção de um compromisso com um toque.
- Cadastro de novo compromisso por um formulário em janela modal.
- Execução no Expo Go e no navegador.

Parte	Conteúdo
0	Preparação do monitor e conceitos básicos.
1	Verificação e atualização do ambiente.
2	Criação do projeto Expo SDK 54.
3	Primeira execução no celular e na web.
4	Planejamento do aplicativo de agenda.
5	Configuração da navegação.
6	Construção passo a passo do aplicativo.
7	Testes orientados.
8	Solução de erros comuns.
9	Atividades de consolidação.
Apêndices	Código completo para conferência.

Parte 0 — Preparação do monitor

0.1 O que o aluno precisa saber antes de começar?

Não é necessário que o aluno saiba React Native. Contudo, o monitor deve apresentar rapidamente os conceitos abaixo antes da prática.

Termo	Explicação simples
Node.js	Programa que permite executar ferramentas JavaScript no computador.
npm	Gerenciador que instala bibliotecas e ferramentas usadas pelo projeto.
React Native	Tecnologia para criar interfaces de aplicativos usando JavaScript ou TypeScript.
Expo	Conjunto de ferramentas que facilita criar, executar e testar projetos React Native.
Expo Go	Aplicativo instalado no celular para abrir o projeto durante a aprendizagem.
TypeScript	JavaScript com tipos, usado para detectar erros e deixar o código mais claro.

Fala sugerida "Hoje não vamos decorar comandos. Vamos aprender um fluxo: criar o projeto, abrir os arquivos, alterar o código, salvar e observar a mudança no celular ou no navegador."

0.2 Conhecimentos praticados

- Componente: uma parte da interface, como uma tela, botão ou cartão de compromisso.
- Estado: uma informação que muda durante o uso, como a lista de compromissos.
- Evento: uma ação do usuário, como tocar em um compromisso ou salvar o formulário.
- Array: uma lista; neste projeto, a lista de compromissos.
- Condição: uma decisão do programa, como verificar se os campos foram preenchidos.
- Renderização: o processo de apresentar os dados na tela.

0.3 Checklist do monitor antes da aula

- ☐ Testar o tutorial em um computador semelhante ao usado pela turma.
- ☐ Confirmar acesso à internet para instalar o projeto e as dependências.
- ☐ Confirmar que o Node.js LTS está instalado.
- ☐ Confirmar que o npm não está na versão 12 enquanto a incompatibilidade descrita neste material persistir.
- ☐ Instalar ou disponibilizar o Visual Studio Code.
- ☐ Pedir aos alunos que instalem o Expo Go antes da aula.
- ☐ Confirmar que computador e celular podem usar a mesma rede Wi-Fi.
- ☐ Separar uma alternativa: navegador web, caso o celular não consiga abrir o QR Code.
- ☐ Ter uma cópia do código completo disponível para comparação.

☐ Projetar o terminal e aumentar o tamanho da fonte para facilitar a leitura.

Parte 1 — Verificação e atualização do ambiente

Nota de compatibilidade Este material usa explicitamente o Expo SDK 54 para compatibilidade com o Expo Go em dispositivo físico e também permite executar o mesmo projeto na web.

1. Abrir o terminal

No Windows, o aluno pode abrir o PowerShell. No macOS, pode abrir o Terminal. No Linux, pode usar o terminal padrão da distribuição.

Explique antes de digitar O terminal é uma janela em que damos instruções ao computador por meio de comandos. Cada comando deve ser digitado e confirmado com a tecla Enter.

2. Verificar Node.js e npm

Digite os dois comandos, um de cada vez:

```
node --version  
npm --version
```

O primeiro comando mostra a versão do Node.js. O segundo mostra a versão do npm. Para o Expo SDK 54, utilize Node.js LTS com versão mínima 20.19.x. Uma versão LTS mais nova também pode ser usada, desde que o npm seja compatível com o criador do projeto.

Resultado esperado Os comandos devem apresentar números de versão. Exemplo: Node.js v22.x e npm 10.x ou 11.x.

Comando não reconhecido Se aparecer "node não é reconhecido", "command not found" ou mensagem semelhante, o Node.js não está instalado ou o terminal ainda não reconheceu a instalação. Instale a versão LTS, feche todas as janelas do terminal, abra novamente e repita os comandos.

3. Evitar a incompatibilidade do npm 12

Há um problema conhecido em que determinadas versões do create-expo-app não conseguem interpretar a saída do comando npm pack quando o computador utiliza npm 12. O erro apresenta a mensagem "Could not parse JSON returned from npm pack".

Se o comando npm --version começar com 12, instale temporariamente a versão 11:

```
npm install --global npm@11  
npm --version
```

Feche e abra o terminal após a instalação. O resultado do segundo comando deve começar com 11.

Importante Não use sudo automaticamente e não altere permissões do sistema sem necessidade. Em computadores institucionais, solicite apoio do setor responsável caso a instalação global seja bloqueada.

4. Verificar o acesso ao registro do npm

```
npm config get registry
```

O resultado esperado é:

```
https://registry.npmjs.org/
```

Se aparecer outro endereço e a instituição não utiliza um registro próprio, configure o registro oficial:

```
npm config set registry https://registry.npmjs.org/  
npm ping
```

5. Verificar o criador de projetos

```
npx create-expo-app@latest --help
```

Na primeira execução, o npm pode perguntar se deseja instalar o pacote create-expo-app. Digite y e pressione Enter. O objetivo deste passo é apenas verificar se a ferramenta abre a tela de ajuda.

Não instale o antigo expo-cli global Use os comandos iniciados por npx. O Expo CLI atual é instalado dentro do projeto com o pacote expo.

1.1 Diagnóstico rápido antes de prosseguir

- ☐ node --version apresenta um número.
- ☐ npm --version apresenta versão 10 ou 11.
- ☐ npm config get registry apresenta o registro correto.
- ☐ npx create-expo-app@latest --help funciona.
- ☐ O aluno sabe em qual pasta deseja criar o projeto.

Parte 2 — Criação do projeto com Expo SDK 54

1. Escolher uma pasta de trabalho

Peça aos alunos que criem uma pasta simples para atividades. Evite nomes com muitos espaços, acentos ou símbolos. Um exemplo adequado é "projetos".

No terminal, navegue até a pasta desejada. Exemplo no Windows, caso a pasta esteja na Área de Trabalho:

```
cd $HOME\Desktop\projetos
```

Exemplo no macOS ou Linux:

```
cd ~/Desktop/projetos
```

Para iniciantes Se a turma tiver dificuldade com o comando `cd`, o monitor pode abrir a pasta no Explorador/Finder e usar a opção "Abrir no terminal", quando disponível.

2. Criar o projeto explicitamente com SDK 54

```
npx create-expo-app@latest agenda-app --template default@sdk-54
```

O comando faz quatro coisas:

- Cria uma pasta chamada `agenda-app`.
- Baixa o template padrão do Expo SDK 54.
- Configura TypeScript e Expo Router.
- Instala as dependências necessárias para Android, iOS e web.

Aguarde a conclusão A instalação pode demorar alguns minutos. Não feche o terminal enquanto os pacotes estiverem sendo baixados.

3. Entrar na pasta do projeto

```
cd agenda-app
```

A partir deste ponto, todos os comandos do projeto devem ser executados dentro dessa pasta.

4. Abrir no Visual Studio Code

```
code .
```

O ponto significa "a pasta atual". Se o comando `code` não funcionar, abra o Visual Studio Code manualmente e escolha Arquivo > Abrir Pasta > `agenda-app`.

5. Remover o exemplo inicial do template

```
npm run reset-project
```

O script move os arquivos de demonstração para uma pasta de exemplo e deixa a pasta `app` com os dois arquivos usados neste tutorial:

```
app/  
├── layout.tsx  
└── index.tsx
```

Explique a estrutura A pasta app contém as telas. O arquivo index.tsx será a tela principal. O arquivo _layout.tsx controla como as telas são organizadas.

6. Verificar a versão instalada

```
npm list expo react react-native
```

Na saída, a versão do pacote expo deve começar com 54. Os números completos podem variar conforme as correções publicadas para o SDK.

7. Verificar a saúde do projeto

```
npx expo install --check  
npx expo-doctor@latest
```

O primeiro comando verifica se as bibliotecas estão nas versões compatíveis. O segundo analisa problemas comuns do projeto.

Se o Expo indicar dependências incompatíveis, execute:

```
npx expo install --fix  
npx expo-doctor@latest
```

Evite npm update Não use npm update para atualizar tudo indiscriminadamente. Em projetos React Native, as versões de algumas bibliotecas precisam permanecer alinhadas ao SDK.

Parte 3 — Primeira execução no celular e na web

1. Iniciar o servidor de desenvolvimento

```
npx expo start --clear
```

A opção `--clear` limpa o cache do Metro Bundler. Na primeira aula, isso reduz a chance de o aluno visualizar arquivos antigos ou receber erros de cache.

2. Abrir no Expo Go

- Confirme que o Expo Go está instalado no celular.
- Confirme que o computador e o celular estão na mesma rede Wi-Fi.
- No Android, abra o Expo Go e use a opção de ler o QR Code.
- No iPhone, use a câmera do sistema para ler o QR Code.
- Aguarde o carregamento do projeto.

3. Abrir no navegador

Com o servidor aberto, clique no terminal e pressione a tecla:

```
w
```

O navegador deve abrir automaticamente. Essa opção é útil quando a rede bloqueia a comunicação com o celular.

4. Parar e reiniciar

Para encerrar o servidor, pressione:

```
Ctrl + C
```

Para iniciar novamente:

```
npx expo start --clear
```

Pausa pedagógica Antes de escrever o aplicativo, peça aos alunos que alterem um texto simples no arquivo `app/index.tsx`, salvem e observem a atualização. Isso demonstra o ciclo editar > salvar > visualizar.

Parte 4 — Planejamento do aplicativo de Agenda

4.1 Regras do aplicativo

- A tela mostra uma lista de compromissos.
- Cada compromisso tem título, data, horário e categoria.
- O jogador (usuário) toca na caixa de seleção para marcar um compromisso como concluído.
- Um botão "×" remove o compromisso da lista.
- Um botão "+ Novo compromisso" abre um formulário em janela modal.
- O formulário exige título, data e horário preenchidos para salvar.
- A lista é reordenada automaticamente por data e horário após cada cadastro.
- O placar mostra o total de compromissos, quantos foram concluídos e o horário do próximo pendente.

4.2 Informações que mudam durante o uso

Informação	Por que precisa ser armazenada?
Lista de compromissos	Para saber quais existem, quais foram concluídos e em que ordem aparecem.
Visibilidade do modal	Para abrir e fechar o formulário de cadastro.
Título digitado	Para montar o novo compromisso ao salvar.
Data digitada	Para ordenar a lista e exibir no cartão.
Horário digitado	Para ordenar a lista e exibir no cartão.
Categoria selecionada	Para colorir o cartão e o rótulo do compromisso.

4.3 Formato de um compromisso

```
type Category = 'trabalho' | 'estudo' | 'pessoal' | 'saude';

type Appointment = {
  id: number;
  title: string;
  date: string;
  time: string;
  category: Category;
  done: boolean;
};
```

Propriedade	Significado
id	Identificador único do compromisso.
title	Nome ou descrição do compromisso.
date	Data apresentada no cartão, no formato dia/mês.
time	Horário apresentado no cartão, no formato hora:minuto.
category	Um entre quatro tipos fixos: trabalho, estudo, pessoal ou saude.

done	Indica se o compromisso já foi concluído.
------	---

Parte 5 — Configuração da tela

1. Editar app/_layout.tsx

Abra o arquivo app/_layout.tsx, apague o conteúdo e coloque:

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerShown: false,
      }}
    />
  );
}
```

O Stack organiza as telas do aplicativo. A opção headerShown: false remove o cabeçalho automático para que a interface da agenda use todo o espaço disponível.

Verificação Salve o arquivo. Se aparecer um erro, confira parênteses, chaves, aspas e ponto e vírgula.

Parte 6 — Construção do aplicativo

Estratégia de aula Nesta parte, construa o arquivo `app/index.tsx` em blocos. Depois de cada bloco, pare para explicar o objetivo. O código completo está no apêndice para conferência.

6.1 Preparar o arquivo e os imports

Abra `app/index.tsx`, apague o conteúdo e comece com os imports:

```
import { useMemo, useState } from 'react';
import {
  FlatList,
  Modal,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native';
import { StatusBar } from 'expo-status-bar';
```

- `useState`: guarda valores que mudam, como a lista e os campos do formulário.
- `useMemo`: recalcula um valor somente quando algo do qual ele depende muda, como o total de concluídos.
- `FlatList`: mostra a lista de compromissos.
- `Pressable`: recebe o toque do usuário nos botões e cartões.
- `Modal`: apresenta o formulário de novo compromisso.
- `TextInput`: campo de texto do formulário.
- `ScrollView`: permite rolar a tela em dispositivos pequenos.
- `StyleSheet`: organiza os estilos.

6.2 Criar o tipo do compromisso e as categorias

```
type Category = 'trabalho' | 'estudo' | 'pessoal' | 'saude';

type Appointment = {
  id: number;
  title: string;
  date: string;
  time: string;
  category: Category;
  done: boolean;
};
```

O TypeScript usa esse tipo para verificar se todo compromisso possui as seis informações necessárias.

6.3 Definir rótulos, cores e constantes

```
const CATEGORY_LABELS: Record<Category, string> = {
  trabalho: 'Trabalho',
  estudo: 'Estudo',
  pessoal: 'Pessoal',
  saude: 'Saúde',
};

const CATEGORY_COLORS: Record<Category, string> = {
  trabalho: '#2563eb',
  estudo: '#7c3aed',
};
```

```

    pessoal: '#0d9488',
    saude: '#dc2626',
  };

  const CATEGORY_OPTIONS: Category[] = ['trabalho', 'estudo', 'pessoal', 'saude'];

  let nextId = 4;

```

`Record<Category, string>` garante que exista um rótulo e uma cor para cada uma das quatro categorias. A variável `nextId` controla o identificador do próximo compromisso cadastrado.

6.4 Criar a lista inicial de compromissos

```

const INITIAL_APPOINTMENTS: Appointment[] = [
  {
    id: 1,
    title: 'Reunião com orientador',
    date: '29/07',
    time: '09:00',
    category: 'trabalho',
    done: false,
  },
  {
    id: 2,
    title: 'Estudar para a prova de redes',
    date: '29/07',
    time: '14:30',
    category: 'estudo',
    done: false,
  },
  {
    id: 3,
    title: 'Consulta com dentista',
    date: '30/07',
    time: '11:00',
    category: 'saude',
    done: false,
  },
];

```

Essa lista serve apenas para a tela não começar vazia durante a demonstração em aula.

6.5 Ordenar os compromissos

```

function sortAppointments(list: Appointment[]): Appointment[] {
  return [...list].sort((a, b) => {
    if (a.date === b.date) {
      return a.time.localeCompare(b.time);
    }
    return a.date.localeCompare(b.date);
  });
}

```

O operador `...` copia a lista antes de ordenar, para não alterar o array original. Quando duas datas são iguais, o horário decide a ordem; caso contrário, a data decide.

6.6 Criar o componente principal e os estados

```

export default function Index() {
  const [appointments, setAppointments] = useState<Appointment[]>(() =>
    sortAppointments(INITIAL_APPOINTMENTS),
  );
  const [isModalVisible, setIsModalVisible] = useState(false);
  const [title, setTitle] = useState('');
  const [date, setDate] = useState('');
  const [time, setTime] = useState('');
}

```

```
const [category, setCategory] = useState<Category>('trabalho');
```

A função `useState` devolve duas partes: o valor atual e uma função para alterar esse valor. Os quatro últimos estados guardam o que o usuário digita no formulário antes de salvar.

Pergunta para a turma "Qual estado muda quando um novo compromisso é salvo?"

Resposta esperada: `appointments`. Os campos `title`, `date`, `time` e `category` também são limpos em seguida.

6.7 Calcular o placar com `useMemo`

```
const totalCount = appointments.length;
const doneCount = useMemo(
  () => appointments.filter((item) => item.done).length,
  [appointments],
);
const nextAppointment = useMemo(
  () => appointments.find((item) => !item.done),
  [appointments],
);
```

`filter` cria uma lista apenas com os compromissos concluídos, e `length` conta quantos existem. `find` procura o primeiro compromisso ainda não concluído, que será exibido como "próximo".

6.8 Abrir e fechar o formulário

```
function openNewAppointmentModal() {
  setTitle('');
  setDate('');
  setTime('');
  setCategory('trabalho');
  setIsModalVisible(true);
}

function closeModal() {
  setIsModalVisible(false);
}
```

Antes de abrir o formulário, os campos são limpos para que o cadastro anterior não apareça na próxima abertura.

6.9 Salvar um novo compromisso

```
function handleSaveAppointment() {
  if (title.trim() === '' || date.trim() === '' || time.trim() === '') {
    return;
  }

  const newAppointment: Appointment = {
    id: nextId,
    title: title.trim(),
    date: date.trim(),
    time: time.trim(),
    category,
    done: false,
  };
  nextId += 1;

  setAppointments((currentAppointments) =>
    sortAppointments([...currentAppointments, newAppointment]),
  );
  setIsModalVisible(false);
}
```

trim() remove espaços em branco no início e no fim do texto. A barreira de segurança impede salvar um compromisso sem título, data ou horário. Depois de montar o novo compromisso, a lista é reordenada e o modal é fechado.

6.10 Marcar como concluído e remover

```
function toggleDone(id: number) {
  setAppointments((currentAppointments) =>
    currentAppointments.map((item) =>
      item.id === id ? { ...item, done: !item.done } : item,
    ),
  );
}

function removeAppointment(id: number) {
  setAppointments((currentAppointments) =>
    currentAppointments.filter((item) => item.id !== id),
  );
}
```

map percorre a lista inteira e altera apenas o compromisso com o id tocado, invertendo o valor de done. filter devolve uma nova lista sem o compromisso removido.

6.11 Criar a interface: cabeçalho e placar

```
return (
  <View style={styles.screen}>
    <StatusBar style="dark" />
    <ScrollView
      contentContainerStyle={styles.screenContent}
      showsVerticalScrollIndicator={false}
    >
      <View style={styles.header}>
        <Text style={styles.title}>Minha Agenda</Text>
        <Text style={styles.subtitle}>Organize seus compromissos do dia</Text>
      </View>

      <View style={styles.scoreboard}>
        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Total</Text>
          <Text style={styles.scoreValue}>{totalCount}</Text>
        </View>
        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Concluídos</Text>
          <Text style={styles.scoreValue}>{doneCount}</Text>
        </View>
        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Próximo</Text>
          <Text style={styles.scoreValueSmall}>
            {nextAppointment ? nextAppointment.time : '--:--'}
          </Text>
        </View>
      </View>
    </View>
  )
```

O placar usa o operador ternário para mostrar "--:--" quando não existe mais nenhum compromisso pendente.

6.12 Criar a lista de compromissos

```
<FlatList
  data={appointments}
  keyExtractor={(item) => String(item.id)}
  scrollEnabled={false}
  contentContainerStyle={styles.list}
  ListEmptyComponent={
```



```

    <Text style={styles.emptyText}>
      Nenhum compromisso cadastrado. Toque em "Novo compromisso" para
      começar.
    </Text>
  }
  renderItem={({ item }) => (
    <View
      style={[
        styles.card,
        item.done && styles.cardDone,
        { borderLeftColor: CATEGORY_COLORS[item.category] },
      ]}
    >
      <Pressable
        accessibilityRole="checkbox"
        accessibilityState={{ checked: item.done }}
        onPress={() => toggleDone(item.id)}
        style={styles.cardMain}
      >
        <View style={[styles.checkbox, item.done && styles.checkboxChecked]}>
          {item.done && <Text style={styles.checkboxMark}></Text>}
        </View>
        <View style={styles.cardInfo}>
          <Text style={styles.cardTitle, item.done && styles.cardTitleDone}>
            {item.title}
          </Text>
          <Text style={styles.cardMeta}>{item.date} às {item.time}</Text>
          <View style={styles.categoryTag, { backgroundColor:
CATEGORY_COLORS[item.category] }}>
            <Text
style={styles.categoryTagText}>{CATEGORY_LABELS[item.category]}</Text>
          </View>
        </View>
      </Pressable>
      <Pressable
        accessibilityRole="button"
        accessibilityLabel={`Remover ${item.title}`}
        onPress={() => removeAppointment(item.id)}
        style={({ pressed }) => [styles.removeButton, pressed &&
styles.removeButtonPressed]}
      >
        <Text style={styles.removeButtonText}>x</Text>
      </Pressable>
    </View>
  )}
/>

```

ListEmptyComponent mostra uma mensagem quando a lista está vazia. Dentro de cada cartão, o estilo muda de acordo com item.done e com a cor da categoria, usando um array de estilos condicionais.

6.13 Criar o botão de novo compromisso e o formulário modal

```

    <Pressable
      accessibilityRole="button"
      onPress={openNewAppointmentModal}
      style={({ pressed }) => [styles.addButton, pressed &&
styles.addButtonPressed]}
    >
      <Text style={styles.addButtonText}>+ Novo compromisso</Text>
    </Pressable>
  </ScrollView>

  <Modal visible={isModalVisible} transparent animationType="fade"
onRequestClose={closeModal}>
    <View style={styles.modalOverlay}>
      <View style={styles.modalContent}>

```

```

        <Text style={styles.modalTitle}>Novo compromisso</Text>

        <Text style={styles.fieldLabel}>Título</Text>
        <TextInput value={title} onChangeText={setTitle} placeholder="Ex: Reunião
de projeto" style={styles.input} />

        <Text style={styles.fieldLabel}>Data</Text>
        <TextInput value={date} onChangeText={setDate} placeholder="Ex: 31/07"
style={styles.input} />

        <Text style={styles.fieldLabel}>Horário</Text>
        <TextInput value={time} onChangeText={setTime} placeholder="Ex: 15:30"
style={styles.input} />

        <Text style={styles.fieldLabel}>Categoria</Text>
        <View style={styles.categoryRow}>
          {CATEGORY_OPTIONS.map((option) => (
            <Pressable
              key={option}
              onPress={() => setCategory(option)}
              style={[
                styles.categoryOption,
                { borderColor: CATEGORY_COLORS[option] },
                category === option && { backgroundColor:
CATEGORY_COLORS[option] },
              ]}
            >
              <Text style={[styles.categoryOptionText, category === option &&
styles.categoryOptionTextActive]}>
                {CATEGORY_LABELS[option]}
              </Text>
            </Pressable>
          ))}
        </View>

        <View style={styles.modalActions}>
          <Pressable onPress={closeModal} style={{ pressed }} =>
[styles.cancelButton, pressed && styles.cancelButtonPressed]]>
            <Text style={styles.cancelButtonText}>Cancelar</Text>
          </Pressable>
          <Pressable onPress={handleSaveAppointment} style={{ pressed }} =>
[styles.saveButton, pressed && styles.saveButtonPressed]]>
            <Text style={styles.saveButtonText}>Salvar</Text>
          </Pressable>
        </View>
      </View>
    </View>
  </Modal>
</View>
);
}

```

O formulário usa `CATEGORY_OPTIONS.map` para desenhar um botão para cada categoria automaticamente, sem repetir código quatro vezes. O botão selecionado recebe a cor da categoria como fundo.

6.14 Adicionar os estilos

Abra o apêndice B para copiar o objeto `styles` completo, criado com `StyleSheet.create`. Ele organiza cores, espaçamentos e bordas do cabeçalho, do placar, dos cartões e do formulário modal em um único lugar, seguindo a mesma lógica do CSS.

6.15 Salvar e observar o resultado

- Salve `app/index.tsx`.
- Observe o terminal para verificar erros.

- Aguarde a atualização automática no Expo Go ou no navegador.
- Toque em "Novo compromisso", preencha os campos e confirme se o cartão aparece na lista.
- Toque na caixa de seleção de um compromisso e confirme se o contador de concluídos aumenta.

Parte 7 — Roteiro de testes

Teste	Resultado esperado
Abrir o aplicativo	Três compromissos de exemplo aparecem, ordenados por data e horário.
Tocar na caixa de seleção	O texto do compromisso ganha um traço e o contador de concluídos aumenta em 1.
Tocar novamente na caixa de seleção	O compromisso volta ao estado pendente.
Tocar no "X" de um compromisso	O compromisso desaparece da lista imediatamente.
Tocar em "Novo compromisso" sem preencher nada	O formulário abre, mas tocar em Salvar não faz nada.
Preencher título, data e horário e tocar em Salvar	Um novo cartão aparece na posição correta da lista.
Remover todos os compromissos	A mensagem de lista vazia aparece no lugar dos cartões.
Abrir no navegador	O layout permanece centralizado e responsivo.

7.1 Técnica de teste para o monitor

Para testar rapidamente o placar "Próximo", o monitor pode marcar todos os compromissos de exemplo como concluídos e depois cadastrar um novo, observando o horário aparecer no lugar de "--:--".

Parte 8 — Solução de erros comuns

8.1 Erro ao criar o projeto: Could not parse JSON returned from npm pack

Causa provável O npm 12 alterou o formato da resposta usada por versões do create-expo-app. Enquanto a ferramenta não estiver ajustada no ambiente utilizado, use npm 11.

```
node --version
npm --version
npm install --global npm@11
```

Feche o terminal, abra novamente e confirme:

```
npm --version
```

Exclua a pasta incompleta antes de tentar novamente.

Windows PowerShell:

```
Remove-Item -Recurse -Force .\agenda-app -ErrorAction SilentlyContinue
```

macOS ou Linux:

```
rm -rf agenda-app
```

Verifique o cache e recrie com SDK 54:

```
npm cache verify
npx create-expo-app@latest agenda-app --template default@sdk-54
```

8.2 node ou npm não é reconhecido

- Instale o Node.js LTS.
- Feche e abra o terminal.
- Reinicie o computador se necessário.
- Repita node --version e npm --version.

8.3 O projeto foi criado, mas o terminal está na pasta errada

Verifique o conteúdo da pasta:

Windows:

```
dir
```

macOS ou Linux:

```
ls
```

A pasta correta deve possuir package.json. Entre nela usando cd agenda-app.

8.4 Erro vermelho depois de editar o código

Leia a primeira mensagem do terminal ou da tela. Erros frequentes:

Mensagem ou sintoma	O que conferir
Unexpected token	Chave, parêntese, colchete, vírgula ou aspas ausentes.
Cannot find name	Nome digitado diferente da declaração.

Mensagem ou sintoma	O que conferir
Cannot find module	Import incorreto ou dependências ainda não instaladas.
Tela branca	Erro no terminal, return incompleto ou componente não fechado.
Text strings must be rendered	Texto colocado fora de um componente Text.
Property does not exist on type	Categoria digitada fora das quatro opções de Category.

Estratégia de depuração Compare somente o bloco que acabou de ser digitado. Não substitua o arquivo inteiro sem antes tentar localizar o caractere ou nome incorreto.

8.5 Alterações não aparecem

```
npx expo start --clear
```

- Confirme que o arquivo foi salvo.
- Confirme que o servidor está aberto na pasta correta.
- Pressione r no terminal para recarregar.
- Feche e reabra o aplicativo Expo Go.

8.6 O celular não abre o QR Code

- Confirme que computador e celular estão na mesma rede.
- Desative temporariamente VPNs que isolem a conexão.
- Verifique se o firewall permite o Node.js.
- Tente o navegador pressionando w.
- Tente o modo tunnel como alternativa.

```
npx expo start --tunnel
```

Observação O modo tunnel depende de internet e pode iniciar mais lentamente.

8.7 A porta já está em uso

O Expo pode perguntar se deseja usar outra porta. Responda y. Também é possível fechar outros terminais que estejam executando projetos Expo.

8.8 Dependências incompatíveis

```
npx expo install --check
npx expo install --fix
npx expo-doctor@latest
```

Execute os comandos na raiz do projeto, onde está o arquivo package.json.

Parte 9 — Consolidação da aprendizagem

9.1 Perguntas para os alunos

1. Por que cada compromisso precisa de um id diferente?
2. Qual estado guarda os compromissos que aparecem na tela?
3. Por que os campos do formulário são limpos antes de abrir o modal?
4. O que faz o método `map` nesta função `handleCardPress/toggleDone`?
5. O que faz o método `filter` usado em `removeAppointment`?
6. Por que a lista é reordenada depois de cada cadastro?
7. Como o programa impede salvar um compromisso sem título?
8. Qual propriedade muda quando um compromisso é marcado como concluído?

9.2 Pequenos desafios

Desafio	Conceito praticado
Adicionar um campo de "local" ao formulário.	Estado e formulário.
Criar uma quinta categoria, como "lazer".	Tipos e Record.
Ordenar por categoria em vez de data.	Função <code>sort</code> .
Mostrar uma mensagem quando todos os compromissos forem concluídos.	Condições.
Impedir a remoção de um compromisso já concluído.	Eventos e condições.
Trocar as cores das categorias.	Estilos.

9.3 Checklist de encerramento da aula

- ☐ Todos os alunos conseguem iniciar o projeto com `npx expo start`.
- ☐ O projeto abre no Expo Go ou na web.
- ☐ Os compromissos de exemplo aparecem ordenados por data e horário.
- ☐ A caixa de seleção marca e desmarca um compromisso como concluído.
- ☐ O botão "X" remove um compromisso da lista.
- ☐ O formulário modal impede salvar campos vazios.
- ☐ Um novo compromisso aparece na posição correta da lista.
- ☐ Os alunos sabem onde estão os arquivos `_layout.tsx` e `index.tsx`.
- ☐ Os alunos conseguem explicar ao menos estado, evento e condição.

Referências técnicas consultadas

- Expo Documentation - Create your first app
- Expo Documentation - create-expo-app
- Expo Documentation - Tools for development
- Expo Documentation - Expo SDK reference
- Expo Documentation - Upgrade Expo SDK
- React Native Documentation - FlatList e Modal

Data de verificação As orientações de versão e compatibilidade deste material foram verificadas em 29 de julho de 2026. Como ferramentas de desenvolvimento mudam com frequência, consulte a documentação oficial antes de reutilizar o tutorial em turmas futuras.